Pipeline CI/CD intelligent & analyse qualité / risque

TP2 DevOps Avancé — **PayFlex API**

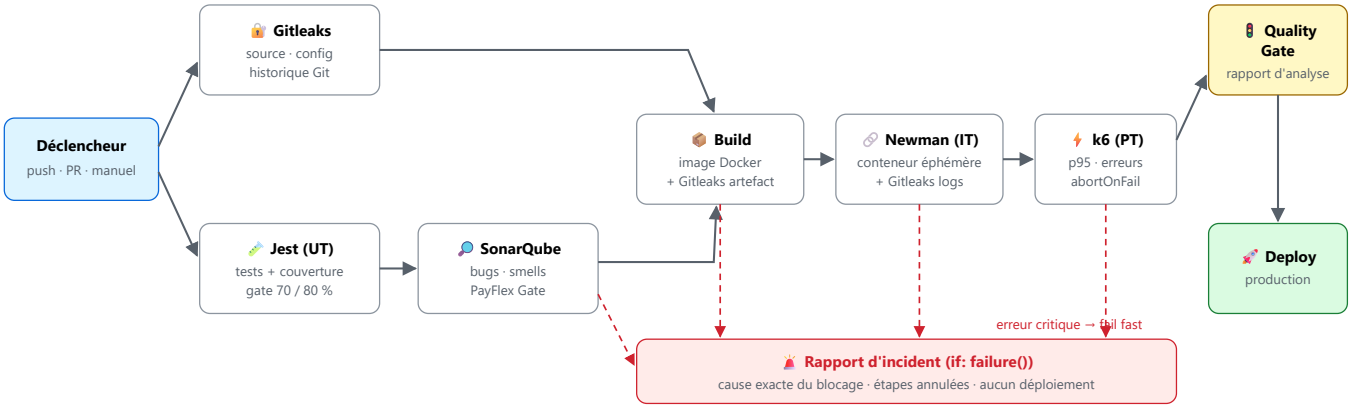
Dali Guerfali · 19 octobre 2026 · Dépôt : **https://github.com/DaliGuerfali/TP2-DevOps-PayFlex**

1. Objectif et choix techniques

PayFlex est une API REST de services financiers (comptes, dépôts, retraits, virements avec frais, validation de carte bancaire). Le pipeline doit garantir qu'aucune régression, fuite de donnée confidentielle ou chute de performance n'atteint la production, et classer automatiquement chaque erreur en **critique** (blocage) ou **non-critique** (alerte).

Catégorie	Outil retenu	Utilisation dans le pipeline
Orchestrateur	GitHub Actions	`.github/workflows/main.yml` : 9 jobs chaînés par `needs`
Application	Node.js 22 + Express	API en mémoire, logs JSON avec masquage des PAN et des champs sensibles
Tests unitaires	Jest + Supertest	103 tests, rapports `lcov` (SonarQube) et `json-summary` (Quality Gate)
Tests d'intégration	Docker + Newman	Image lancée dans un conteneur éphémère, collection Postman de 11 requêtes / 47 assertions
Performance	k6 0.54	20 utilisateurs virtuels pendant 45 s, seuils avec `abortOnFail`
Qualité	SonarQube 10.6 Community	Serveur éphémère (service container), Quality Gate « PayFlex Gate » créée par API
Confidentialité	Gitleaks 8.21	Règles par défaut + règles PII (carte bancaire, IBAN) ; 4 zones scannées

2. Architecture du pipeline



Gitleaks (source) et Jest démarrent en parallèle ; tous les autres jobs attendent la réussite du précédent (`needs`). Une erreur critique fait échouer le job : GitHub Actions marque alors tous les jobs dépendants « skipped », le déploiement n'a jamais lieu et seul le job *Rapport d'incident* s'exécute.

Chaîne de tests multi-niveaux (étape 1)

- **UT** — Jest sur les services métier (frais, Luhn, comptes, virements), les utilitaires (masquage, logger, config) et l'API HTTP (Supertest). Couverture obtenue : **100 % des lignes**.
- **IT** — l'image Docker produite par le build est chargée et lancée dans un conteneur éphémère ; Newman exécute le scénario complet : santé, ouverture de deux comptes, dépôt, virement avec frais, contrôle du flux de données (historique du bénéficiaire), règles métier (422, 400, 404) et validation de carte (le PAN complet ne doit jamais être renvoyé).
- **PT** — k6 : montée à 20 VUs en 10 s, plateau 30 s, descente 5 s ; trois requêtes par itération (lecture de compte, virement, validation de carte). Objectif : p95 < 200 ms et erreurs < 1 %.

3. Taux de confidentialité & détection de fuites (étape 2)

Zone scannée	Job	Commande
Historique Git (tous les commits)	secrets-scan	`gitleaks git` avec `fetch-depth: 0`
Code source + fichiers de configuration	secrets-scan	`gitleaks dir .`
Artefact : système de fichiers `/app` de l'image + configuration de l'image (variables d'environnement)	build	`docker export` puis `gitleaks dir artifact-fs`
Logs de l'application après le scénario d'intégration	integration-tests	`docker logs` puis `gitleaks dir`

En plus des règles par défaut de Gitleaks (clés AWS, Stripe, GitHub, clés privées, JWT, clés d'API génériques…), `.gitleaks.toml` ajoute deux règles PII : **numéro de carte bancaire (PAN)** Visa / Mastercard / Amex et **IBAN**. Les numéros de test publics (4111 1111 1111 1111, 4242…, 5555…4444) sont autorisés car ce ne sont pas des données personnelles.

> **Métrique :** Taux de confidentialité = (1 − fuites / max(seuil toléré, 1)) × 100 %, avec un seuil toléré de 0. La formule du sujet divise par zéro quand le seuil vaut 0 ; nous utilisons donc max(seuil, 1) comme dénominateur. Résultat : 0 fuite → 100 % (validé) ; une fuite ou plus → 0 % (pipeline en échec). Le score doit valoir 100 % dans **chacune** des quatre zones.

4. Quality Gate et classification des erreurs (étapes 3 et 4)

Chaque outil produit son rapport brut (Jest, SonarQube, Gitleaks, Newman, k6). Le script `scripts/quality-gate/evaluate.js` le lit, applique les seuils de `quality-gate.config.json` et écrit un indicateur `metrics/*.json` de niveau OK, WARNING ou CRITICAL.

Indicateur	Non-critique (warning)	Critique (pipeline fail)	Source
Couverture de code	70 % ≤ couverture < 80 %	< 70 %	Jest `coverage-summary.json`
Bugs / Smells	code smells, bugs ou vulnérabilités mineurs	bug ou vulnérabilité BLOCKER / CRITICAL, Quality Gate SonarQube en ERROR	API SonarQube
Performance	p95 entre 200 et 500 ms, ou erreurs entre 1 et 2 %	p95 > 500 ms ou erreurs > 2 %	résumé k6
Confidentialité	—	toute fuite (taux < 100 %)	rapports Gitleaks
Intégration	—	au moins une assertion en échec	rapport Newman

La Quality Gate SonarQube « PayFlex Gate » est créée par API au démarrage du serveur éphémère avec les conditions : couverture < 70 % → ERROR, note de fiabilité ou de sécurité D ou E (au moins un bug / une vulnérabilité critique ou bloquante) → ERROR.

Réaction du pipeline

1. **Erreur non-critique.** `evaluate.js` sort avec le code 0 : le job réussit et le pipeline continue. Il émet une annotation GitHub `::warning::` visible sur la page du run (ex. « Attention, couverture à 75 % ») et l'ajoute au résumé du job. Le job final *Quality Gate* liste tous les avertissements et autorise le déploiement « avec avertissements ».
2. **Erreur critique.** `evaluate.js` sort avec le code 1 et émet une annotation `::error::` : le job échoue immédiatement. Tous les jobs suivants sont annulés (*skipped*) grâce aux dépendances `needs`, donc **aucun déploiement**. Pour la performance, k6 lui-même interrompt le tir dès que le seuil critique est franchi (`abortOnFail`). Le job *Rapport d'incident* (`if: failure()`) rassemble les indicateurs déjà produits et publie la cause exacte du blocage, la valeur mesurée, la liste des étapes annulées et le statut de chaque job.
3. **Sécurité par défaut.** Un rapport absent ou illisible (outil planté, fichier manquant) est classé critique : on ne valide jamais un indicateur qu'on n'a pas pu mesurer.

Scénarios de démonstration

Le workflow se lance à la main (*Run workflow*) avec un paramètre `scenario`. Aucun secret n'est versionné : pour `data-leak`, le script `scripts/demo/inject-leak.sh` génère pendant le job une clé Stripe, une clé AWS (id + secret), une clé privée RSA et un numéro de carte valide au sens de Luhn, puis les place dans un fichier de configuration copié dans l'image.

Scénario	Modification	Résultat attendu
`nominal`	aucune	tout vert, déploiement
`perf-warning`	latence simulée de 250 ms par requête	avertissement non-critique, déploiement maintenu
`perf-critical`	latence simulée de 700 ms par requête	k6 interrompu, p95 > 500 ms, pas de déploiement
`data-leak`	secrets et PAN injectés dans l'artefact	taux de confidentialité 0 %, pas de déploiement

5. Résultats des exécutions

Les quatre runs ont été lancés sur le même commit `4512d73`. Synthèse :

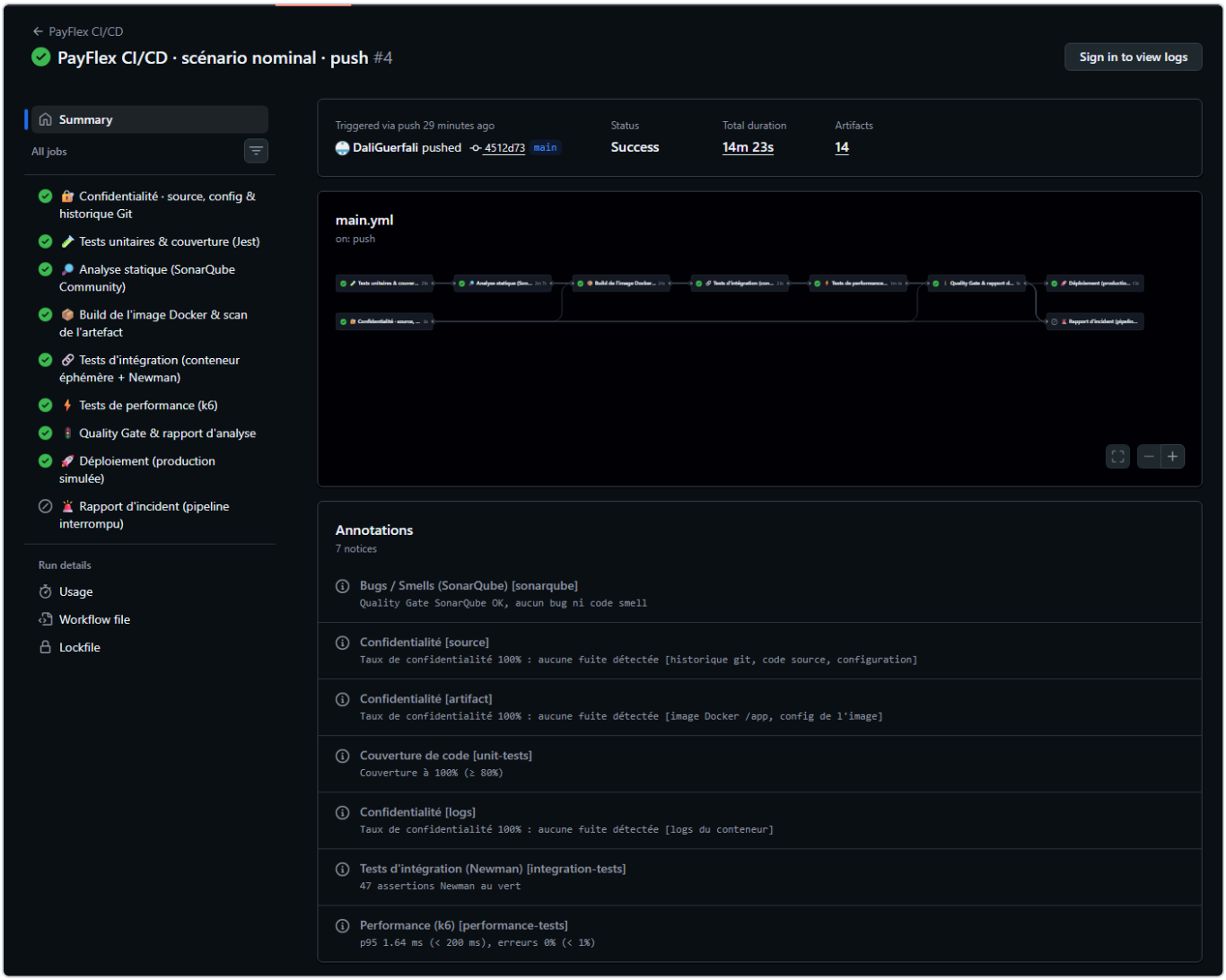
Scénario	Conclusion	Indicateur décisif	Déploiement
`nominal`	Success	7/7 indicateurs OK — p95 1,64 ms sur 11 042 requêtes, couverture 100 %, 0 fuite	effectué
`perf-warning`	Success	WARNING — p95 251,6 ms (objectif < 200 ms)	effectué avec avertissement
`perf-critical`	Failure	CRITICAL — p95 701,95 ms > 500 ms, k6 interrompu après 12 s (134 requêtes au lieu de ~11 000)	annulé
`data-leak`	Failure	CRITICAL — 5 fuites dans l'image Docker, taux de confidentialité 0 %	annulé (IT, PT, gate et déploiement non exécutés)

Analyse SonarQube (run nominal) : Quality Gate « PayFlex Gate » OK — 0 bug, 0 vulnérabilité, 0 code smell, 0 security hotspot, 0 % de duplication, dette technique 0 min, 438 lignes de code, couverture SonarQube 95,2 % (lignes + branches).

5.1 Pipeline réussi — tous les critères au vert

Aucune modification. Les sept indicateurs sont OK, le Quality Gate autorise le déploiement et le job « Déploiement » promeut l'image validée.

Run : **https://github.com/DaliGuerfali/TP2-DevOps-PayFlex/actions/runs/39284227017**



Capture de la page du run GitHub Actions (scénario `nominal`) : statut, graphe des jobs et annotations émises par le Quality Gate.

Rapport produit par le pipeline — artefact `quality-gate-report`

🚦 Rapport du Quality Gate PayFlex

Scénario : `nominal` · Commit : `4512d73`

✅ **DÉPLOIEMENT AUTORISÉ** — tous les critères sont au vert

Indicateurs

Indicateur	Étape	Valeur	Niveau	Détail
Confidentialité	source	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [historique git, code source, configuration]
Couverture de code	unit-tests	100	✅ OK	Couverture à 100% (≥ 80%)

Bugs / Smells (SonarQube)	sonarqube	qualityGate=OK, bugs=0, vulnerabilities=0, codeSmells=0, blockingIssues=0	✅ OK	Quality Gate SonarQube OK, aucun bug ni code smell
Confidentialité	artifact	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [image Docker /app, config de l'image]
Tests d'intégration (Newman)	integration-tests	assertions=47, failedAssertions=0	✅ OK	47 assertions Newman au vert
Confidentialité	logs	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [logs du conteneur]
Performance (k6)	performance-tests	p95Ms=1.64, errorRatePct=0	✅ OK	p95 1.64 ms (< 200 ms), erreurs 0% (< 1%)

Classification des erreurs

Critiques (bloquantes) : 0

Non-critiques (avertissements / dette technique) : 0

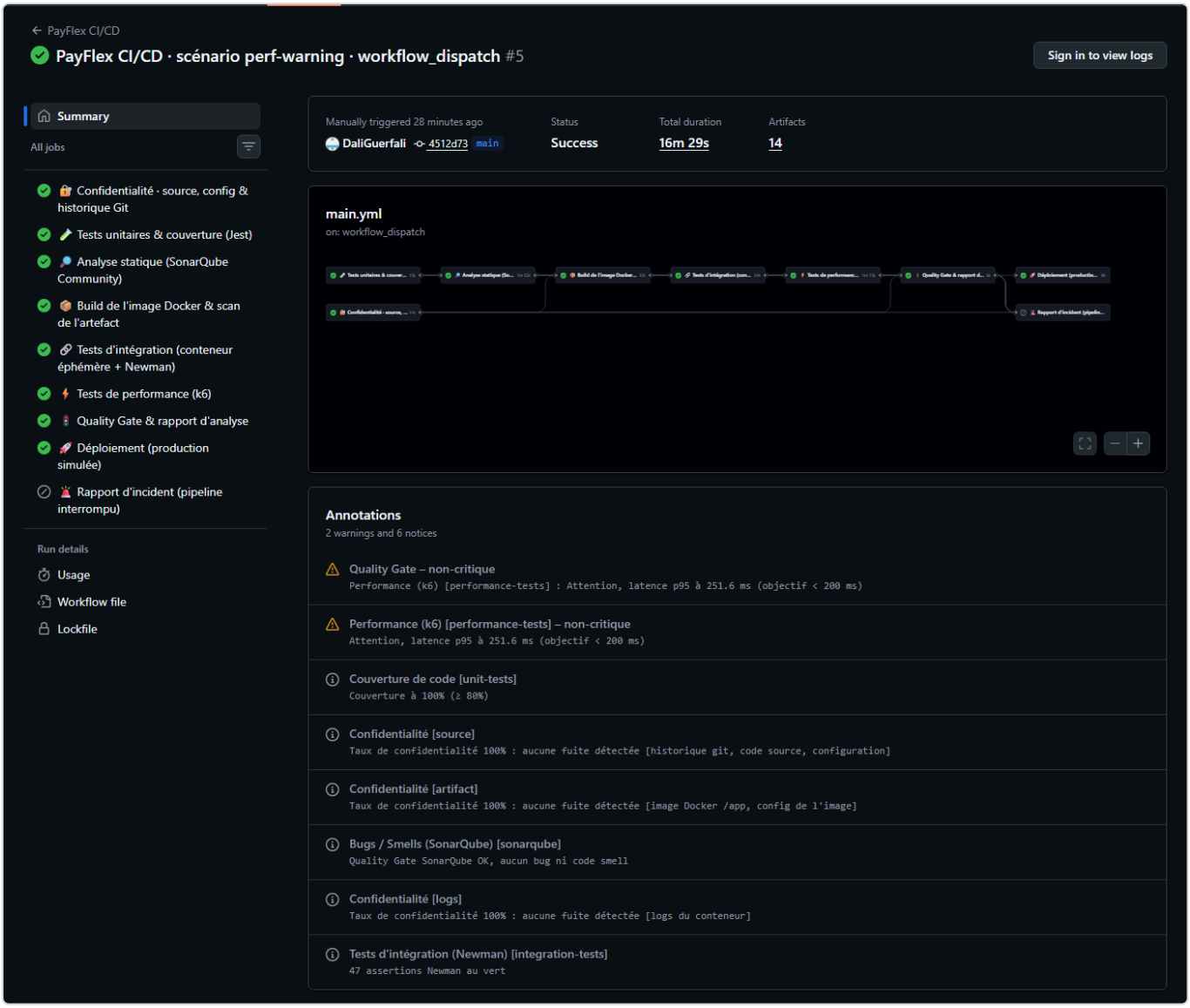
Statut des jobs

Job	Résultat
🔐 Scan des secrets (source, config, historique)	✅ success
🧪 Tests unitaires + couverture	✅ success
🔎 Analyse statique SonarQube	✅ success
📦 Build + scan de l'artefact	✅ success
🔗 Tests d'intégration (Docker + Newman)	✅ success
⚡ Tests de performance (k6)	✅ success

5.2 Erreur non-critique — le pipeline continue avec une alerte

Latence simulée de 250 ms. La p95 (251,6 ms) dépasse l'objectif de 200 ms mais reste sous le seuil critique de 500 ms : le job k6 réussit, une annotation `warning` est émise et le déploiement a lieu.

Run : **https://github.com/DaliGuerfali/TP2-DevOps-PayFlex/actions/runs/39284234445**



Capture de la page du run GitHub Actions (scénario `perf-warning`) : statut, graphe des jobs et annotations émises par le Quality Gate.

Rapport produit par le pipeline — artefact `quality-gate-report`

🚦 Rapport du Quality Gate PayFlex

Scénario : `perf-warning` · Commit : `4512d73`

⚠️ DÉPLOIEMENT AUTORISÉ AVEC 1 AVERTISSEMENT(S)

Indicateurs

Indicateur	Étape	Valeur	Niveau	Détail
Confidentialité	source	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [historique git, code source, configuration]

Couverture de code	unit-tests	100	✅ OK	Couverture à 100% (≥ 80%)
Bugs / Smells (SonarQube)	sonarqube	qualityGate=OK, bugs=0, vulnerabilities=0, codeSmells=0, blockingIssues=0	✅ OK	Quality Gate SonarQube OK, aucun bug ni code smell
Confidentialité	artifact	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [image Docker /app, config de l'image]
Tests d'intégration (Newman)	integration-tests	assertions=47, failedAssertions=0	✅ OK	47 assertions Newman au vert
Confidentialité	logs	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [logs du conteneur]
Performance (k6)	performance-tests	p95Ms=251.6, errorRatePct=0	⚠️ WARNING	Attention, latence p95 à 251.6 ms (objectif < 200 ms)

Classification des erreurs

Critiques (bloquantes) : 0

Non-critiques (avertissements / dette technique) : 1

- Performance (k6) [performance-tests] — Attention, latence p95 à 251.6 ms (objectif < 200 ms)

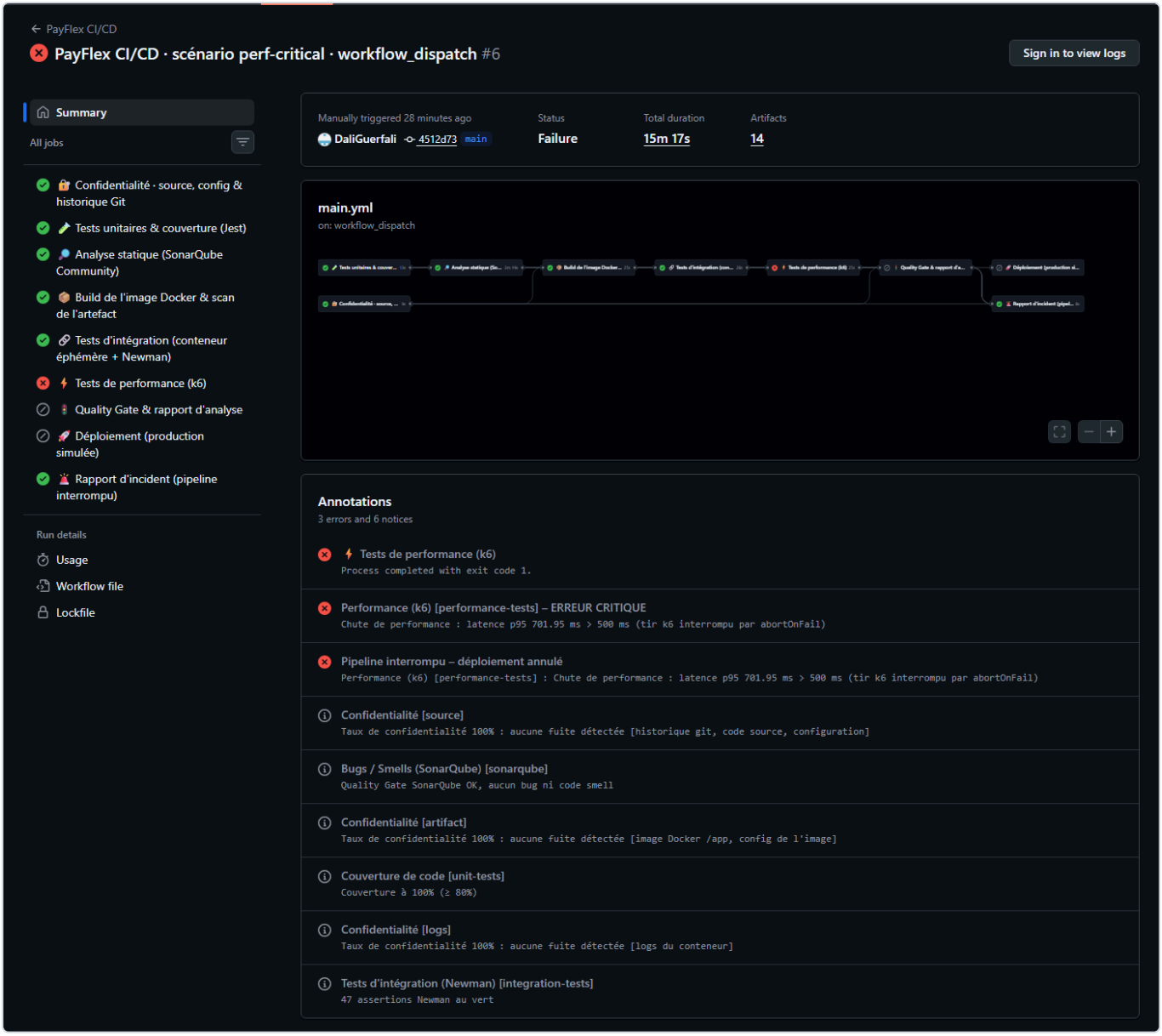
Statut des jobs

Job	Résultat
🔐 Scan des secrets (source, config, historique)	✅ success
🧪 Tests unitaires + couverture	✅ success
🔎 Analyse statique SonarQube	✅ success
📦 Build + scan de l'artefact	✅ success
🔗 Tests d'intégration (Docker + Newman)	✅ success
⚡ Tests de performance (k6)	✅ success

5.3 Pipeline interrompu — test de performance échoué (erreur critique)

Latence simulée de 700 ms. Le seuil `p(95)<500` est franchi : k6 arrête le tir de lui-même (`abortOnFail`) après 12 s, le Quality Gate classe l'indicateur CRITICAL, le job échoue, le Quality Gate final et le déploiement sont annulés et le rapport d'incident donne la cause exacte.

Run : **https://github.com/DaliGuerfali/TP2-DevOps-PayFlex/actions/runs/39284240722**



Capture de la page du run GitHub Actions (scénario `perf-critical`) : statut, graphe des jobs et annotations émises par le Quality Gate.

Rapport produit par le pipeline — artefact `incident-report`

🚨 Rapport d'incident — pipeline interrompu

Scénario : `perf-critical` · Commit : `4512d73`

🛑 **DÉPLOIEMENT BLOQUÉ** — erreur critique détectée (fail fast)

Cause exacte du blocage

- 🛑 **Performance (k6)** (étape `performance-tests`) : Chute de performance : latence p95 701.95 ms > 500 ms (tir k6 interrompu par abortOnFail)

Étapes annulées suite au blocage : `quality-gate` → **aucun déploiement**.

Indicateurs

Indicateur	Étape	Valeur	Niveau	Détail
Confidentialité	source	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [historique git, code source, configuration]
Couverture de code	unit-tests	100	✅ OK	Couverture à 100% (≥ 80%)
Bugs / Smells (SonarQube)	sonarqube	qualityGate=OK, bugs=0, vulnerabilities=0, codeSmells=0, blockingIssues=0	✅ OK	Quality Gate SonarQube OK, aucun bug ni code smell
Confidentialité	artifact	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [image Docker /app, config de l'image]
Tests d'intégration (Newman)	integration-tests	assertions=47, failedAssertions=0	✅ OK	47 assertions Newman au vert
Confidentialité	logs	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [logs du conteneur]
Performance (k6)	performance-tests	p95Ms=701.95, errorRatePct=0	🛑 CRITICAL	Chute de performance : latence p95 701.95 ms > 500 ms (tir k6 interrompu par abortOnFail)

Classification des erreurs

Critiques (bloquantes) : 1

- Performance (k6) [performance-tests] — Chute de performance : latence p95 701.95 ms > 500 ms (tir k6 interrompu par abortOnFail)

Non-critiques (avertissements / dette technique) : 0

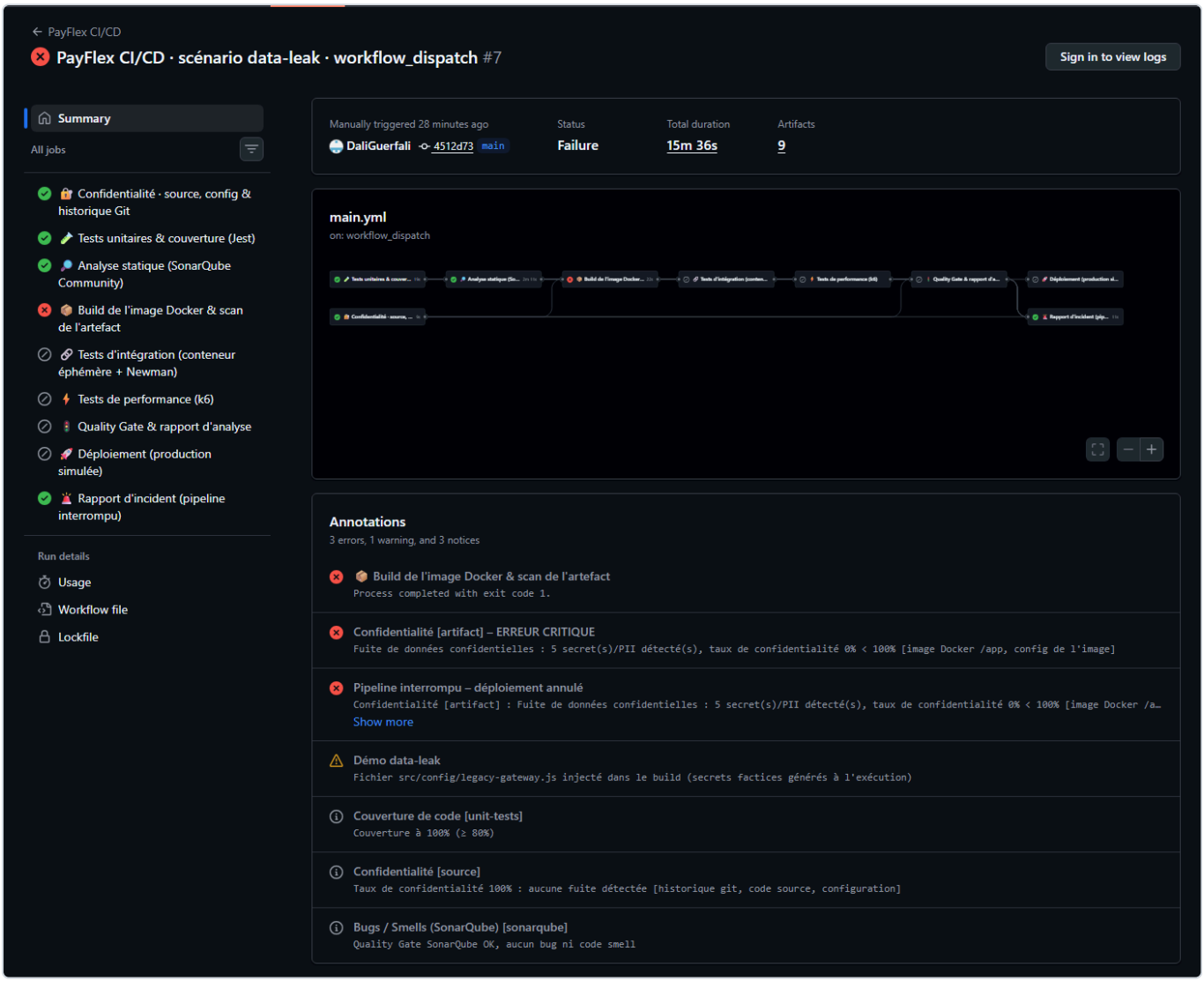
Statut des jobs

Job	Résultat
🔐 Scan des secrets (source, config, historique)	✅ success
🧪 Tests unitaires + couverture	✅ success
🔎 Analyse statique SonarQube	✅ success
📦 Build + scan de l'artefact	✅ success
🔗 Tests d'intégration (Docker + Newman)	✅ success
⚡ Tests de performance (k6)	🛑 failure
🚦 Quality Gate	⏭️ skipped

5.4 Pipeline interrompu — fuite de donnée confidentielle (taux < 100 %)

Un fichier de configuration contenant une clé Stripe, une clé AWS (identifiant + secret), une clé privée RSA et un numéro de carte est ajouté au build. Le code versionné reste propre (taux 100 % sur la source) mais le scan de l'artefact trouve 5 fuites : le taux tombe à 0 % et le pipeline s'arrête dès le job de build — tests d'intégration, performance, Quality Gate et déploiement ne s'exécutent pas.

Run : **https://github.com/DaliGuerfali/TP2-DevOps-PayFlex/actions/runs/39284246639**



Capture de la page du run GitHub Actions (scénario `data-leak`) : statut, graphe des jobs et annotations émises par le Quality Gate.

Rapport produit par le pipeline — artefact `incident-report`

🚨 Rapport d'incident — pipeline interrompu

Scénario : `data-leak` · Commit : `4512d73`

🛑 **DÉPLOIEMENT BLOQUÉ** — erreur critique détectée (fail fast)

Cause exacte du blocage

- 🛑 **Confidentialité** (étape `artifact`) : Fuite de données confidentielles : 5 secret(s)/PII détecté(s), taux de confidentialité 0% < 100% [image Docker /app, config de l'image]

Étapes annulées suite au blocage : `integration-tests`, `performance-tests`, `quality-gate` → **aucun déploiement**.

Indicateurs

Indicateur	Étape	Valeur	Niveau	Détail
Confidentialité	source	leaks=0, ratePct=100	✅ OK	Taux de confidentialité 100% : aucune fuite détectée [historique git, code source, configuration]
Couverture de code	unit-tests	100	✅ OK	Couverture à 100% (≥ 80%)
Bugs / Smells (SonarQube)	sonarqube	qualityGate=OK, bugs=0, vulnerabilities=0, codeSmells=0, blockingIssues=0	✅ OK	Quality Gate SonarQube OK, aucun bug ni code smell
Confidentialité	artifact	leaks=5, ratePct=0	🛑 CRITICAL	Fuite de données confidentielles : 5 secret(s)/PII détecté(s), taux de confidentialité 0% < 100% [image Docker /app, config de l'image]
Tests d'intégration (Newman)	integration-tests	—	⏭️ NOT_RUN	Étape non exécutée
Confidentialité	logs	—	⏭️ NOT_RUN	Étape non exécutée
Performance (k6)	performance-tests	—	⏭️ NOT_RUN	Étape non exécutée

Classification des erreurs

Critiques (bloquantes) : 1

- Confidentialité [artifact] — Fuite de données confidentielles : 5 secret(s)/PII détecté(s), taux de confidentialité 0% < 100% [image Docker /app, config de l'image]

Non-critiques (avertissements / dette technique) : 0

Statut des jobs

Job	Résultat
🔐 Scan des secrets (source, config, historique)	✅ success
🧪 Tests unitaires + couverture	✅ success
🔎 Analyse statique SonarQube	✅ success
📦 Build + scan de l'artefact	🛑 failure
🔗 Tests d'intégration (Docker + Newman)	⏭️ skipped
⚡ Tests de performance (k6)	⏭️ skipped
🚦 Quality Gate	⏭️ skipped

Détail des fuites détectées par Gitleaks dans l'artefact (`reports-build/gitleaks-artifact.json`)

Règle	Description	Fichier dans l'image	Ligne	Valeur
`stripe-access-token`	Found a Stripe Access Token, posing a risk to payment processing services and sensitive financial data.	`/app/src/config/legacy-gateway.js`	3	REDACTED

`aws-access-token`	Identified a pattern that may indicate AWS credentials, risking unauthorized cloud resource access and data breaches on AWS platforms.	`/app/src/config/legacy-gateway.js`	4	REDACTED
`generic-api-key`	Detected a Generic API Key, potentially exposing access to various services and sensitive operations.	`/app/src/config/legacy-gateway.js`	5	REDACTED
`payment-card-number`	Payment card number (PAN) - PII / PCI-DSS data	`/app/src/config/legacy-gateway.js`	6	REDACTED
`private-key`	Identified a Private Key, which may compromise cryptographic security and sensitive data encryption.	`/app/src/config/legacy-gateway.js`	7	REDACTED

Les valeurs sont masquées (`--redact`) : le rapport de détection ne devient pas lui-même une fuite.

6. Conclusion

- Les trois niveaux de tests (UT, IT dans un conteneur éphémère, PT) s'exécutent à chaque push et chaque PR.
- La confidentialité est vérifiée à quatre endroits (historique, source et configuration, artefact, logs) ; une seule fuite suffit à bloquer.
- Toutes les règles de classification sont centralisées dans `quality-gate.config.json` et `scripts/quality-gate/classify.js`, et couvertes par 30 tests unitaires.
- Les erreurs non-critiques sont visibles (annotations, résumé, rapport final) sans bloquer ; les erreurs critiques arrêtent le pipeline au plus tôt et produisent un rapport d'incident.
- Pistes d'amélioration : publier l'image sur un registre (GHCR) au lieu du déploiement simulé, envoyer les alertes vers Slack ou Teams, exporter les indicateurs vers Prometheus / Grafana (bonus observabilité).